Intelligent Tic-Tac-Toe Agents

Jeremy Qiao

University of New Brunswick Saint John
CS4795 - Introduction to AI

Abstract. This project studies how to build an intelligent Tic-Tac-Toe agent and how different intelligent agent methods behave when the board size becomes larger. I studied the basic minimax algorithm, which can always find the best move in small games. I studied alpha-beta pruning, which improves minimax by cutting off unnecessary branches. After that, I looked at depth-limited search with a heuristic evaluation function, which makes the search faster, but the quality of the decision depends heavily on the evaluation rules. Finally, I explored deep learning ideas, including using a convolutional neural network (CNN) as an evaluation function and using a deep neural network (DNN) policy network to predict the next move directly.

This project discovers that minimax is great for small games such as 3×3 boards, but it has high time complexity as the board size becomes larger. Alpha-beta pruning may help reduce the search cost, but it does not fix the exponential growth problem. Heuristic evaluation may help improve speed, but heuristics are handmade and may miss complex board patterns. Using CNN to evaluate the game state allows the model to learn patterns from data, but it still does not reduce the size of the search tree. The DNN policy network allows direct move prediction without a full search, but it will not always give the best result and depends heavily on model design and training data. The project overall shows the trade-off between speed and learning ability in tic tac toe intelligent agent.

1 Introduction

I selected this project because Tic-Tac-Toe provides a simple way to show many basic concepts in artificial intelligence. The game seems simple at first glance, but once the board size increases, it becomes much harder to decide which position to choose next.

This project studies a big question: how can we make intelligent agents better when they cannot check every possible move due to time complexity? A minimax algorithm can check all possible future moves and give the best move based on the current game state. In larger games such as chess, many of these same algorithms are not useful because there is not enough time to check all possible future states. After teh class CS4705, I studied a few ways to improve intelligent agent's performance: alpha-beta pruning, cutoff tests, evaluation functions, convolutional neural networks (CNN) for evaluation, and deep neural networks (DNN) for choosing moves.

Methods can be accurate but may not have good processing time. Then, heuristic methods can improve search time, but designing good heuristics can be difficult. Neural networks can find useful patterns by themselves, while traditional rule-based methods depend on fixed rules.

2 Tic-Tac-Toe

The game Tic-Tac-Toe is played by 2 players who take turns placing either an X or O on a board. The winner is the first player who completes a row, column, or diagonal. When the game board is only 3×3 in size, a computer can calculate all possible future game states and outcomes. However, the number of possibilities increases very quickly when the board size grows to 4×4, 5×5, 6×6, and beyond. The main problem is that there are simply too many possible moves to check.

3 Minimax

Minimax tries to look ahead into the future to determine the outcome of each possible move. There are two players competing: one player wants the highest possible score when they win, and the other wants the lowest. The intelligent agent searches the game tree to find the move that leads to the best final score. At small board sizes, Minimax works well.

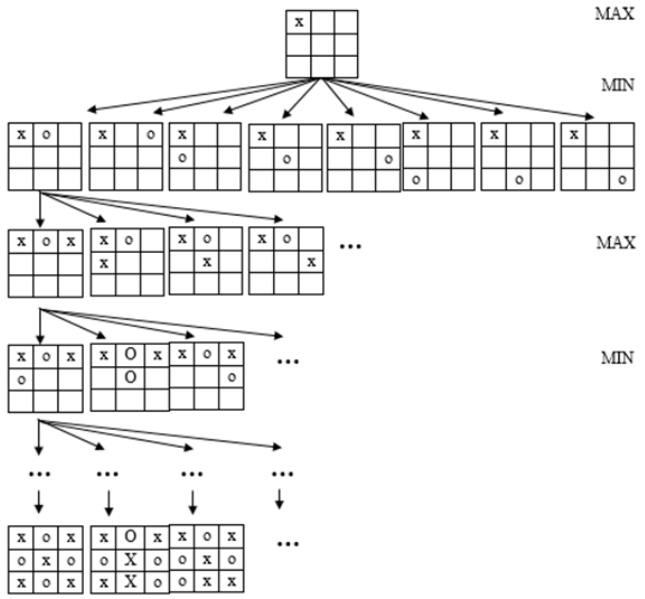

Fig. 1. Minimax game tree example.

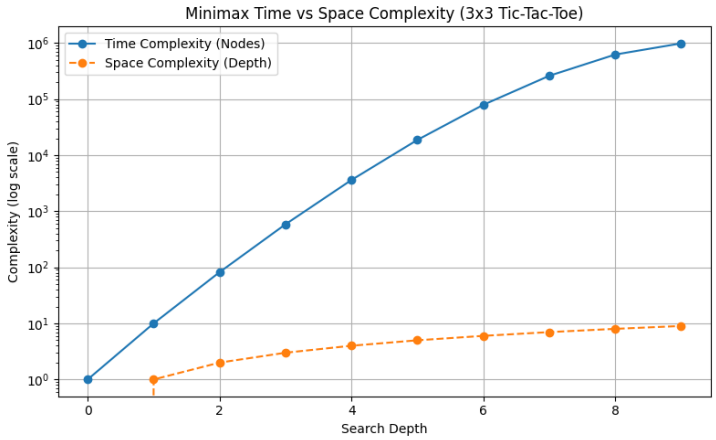

Fig. 2. Minimax time vs space complexity (3×3 Tic-Tac-Toe).

4 Alpha-Beta Pruning

5 Alpha-Beta Pruning

Alpha-Beta Pruning stops searching branches that cannot affect the final result. It removes branches from the minimax tree that neither the min nor max player would ever choose, saving time compared to a full search.

Alpha-Beta Pruning uses two values during the search. Alpha is the lower bound — the best value found so far for the maximizing player. Beta is the upper bound — the best value the minimizing player can enforce. If $\alpha \geq \beta$, the current branch is pruned and the search stops exploring it, because neither player would ever choose that path.

Time complexity improves from $O(b^m)$ to $O(b^{m/2})$ with perfect move ordering, while space complexity remains $O(bm)$ since it still uses depth-first search. However, Alpha-Beta Pruning still has the same fundamental problem of exponential growth for larger boards. Although it is better than standard Minimax, it is still not sufficient for very large boards.

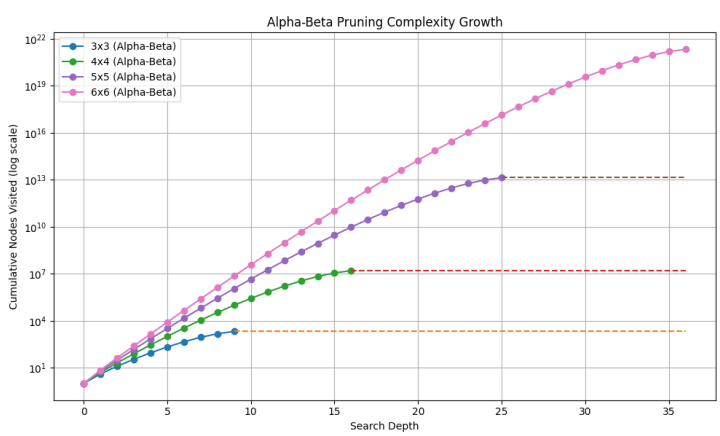

Fig. 3. Alpha-Beta pruning complexity growth.

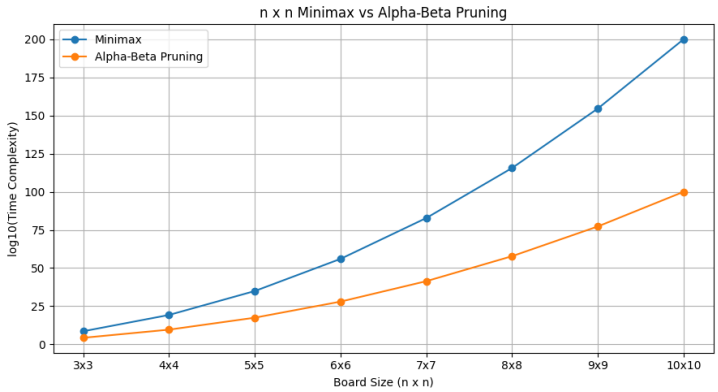

Fig. 4. $n \times n$ Minimax vs Alpha-Beta pruning.

6 Terminal Test and Utility

The terminal test checks whether the current board state is a final game state (a win, loss, or draw). The utility function assigns a numerical value to each terminal state. For Tic-Tac-Toe, a win returns a positive value, a loss returns a negative value, and a draw returns zero. Together, the terminal test and utility function allow Minimax to evaluate the outcome of any complete game sequence.

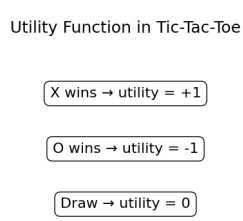

Fig. 5. Utility function in Tic-Tac-Toe.

7 4×4, 5×5, 6×6 ... $n \times n$ Tic-Tac-Toe Agent

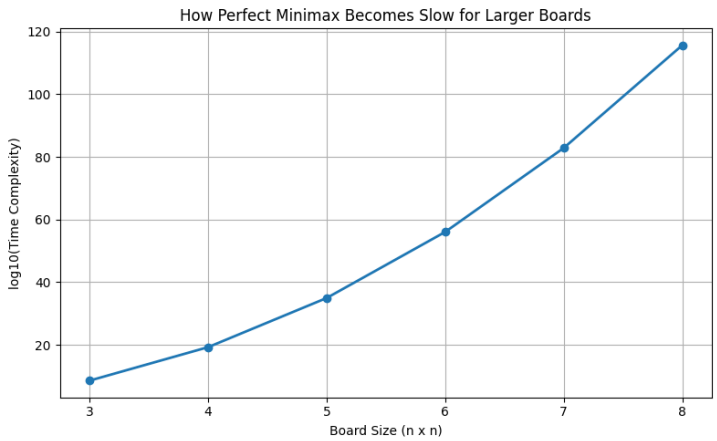

Fig. 6. How perfect Minimax becomes slow for larger boards.

As shown above, Perfect Minimax becomes slow for larger boards. The number of possible game states grows very fast as the board size increases. For a 4×4 board or larger, a perfect Minimax agent cannot finish its search in a reasonable time. This also affects learning-based methods: if Perfect Minimax is used to compute the loss during CNN training, slow evaluation makes it hard to update the model by backpropagation.

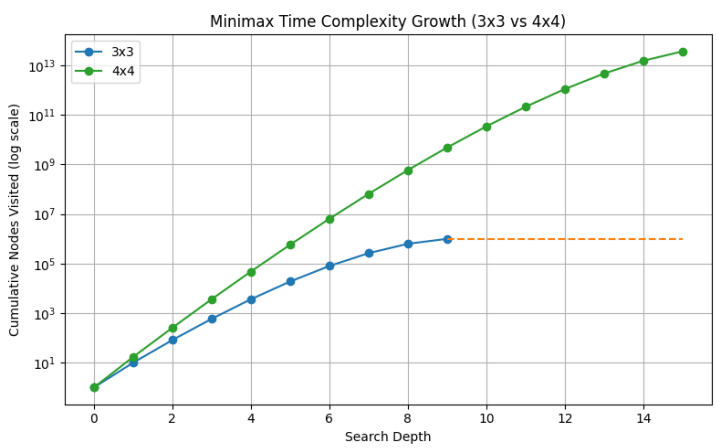

Fig. 7. Minimax time complexity growth (3×3 vs 4×4).

8 Depth-Limited Search

Depth-limited Search stops the game tree search at a set depth and uses an evaluation function to score the board position at that point, rather than searching all the way to a terminal state. The time complexity becomes $O(b^d)$, where b is the branching factor and d is the depth limit. This is faster than full Minimax, but the quality of the result depends on how good the evaluation function is.

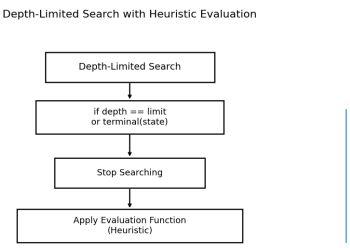

Fig. 8. Depth-limited search with heuristic evaluation.

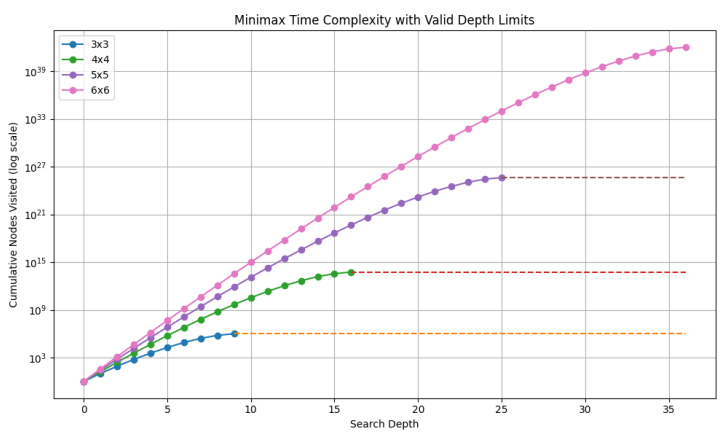

Fig. 9. Minimax time complexity with valid depth limits.

9 Cutoff Test and Evaluation Function (Heuristic)

The cutoff test decides when to stop searching and apply the evaluation function. The evaluation function (heuristic) scores a non-terminal board position as if it

were a terminal state. For example, a simple heuristic might assign +10 if the intelligent agent has two X's in a row, or +3 if the intelligent agent occupies the center. Simple heuristics work on small boards, but as board size grows, the number of strategies increases and simple evaluation rules become ineffective. Heuristic evaluation functions do not "understand" the board, they only follow handcrafted rules (main idea of my report).

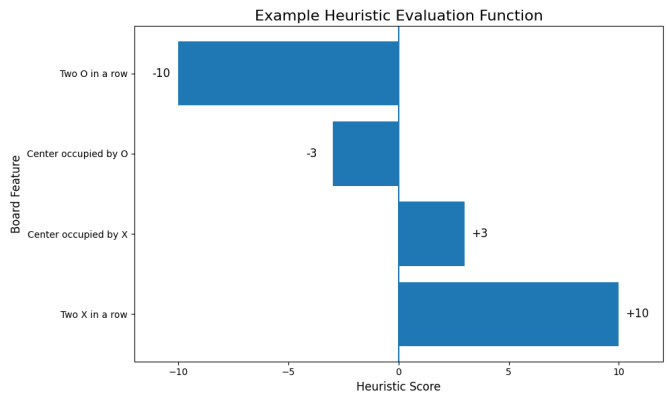

Fig. 10. Example heuristic evaluation function.

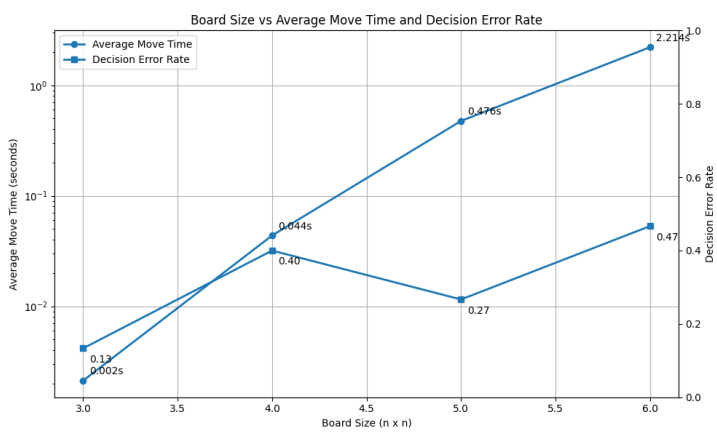

Fig. 11. Board size vs average move time and decision error rate.

As the board gets bigger, the agent becomes slower and starts to make silly decisions. The drop in error rate from 4×4 to 5×5 does not mean the agent got

better. It happens because only 30 board states are tested for each size, so the numbers can go up or down by chance. The heuristic also works better on some board setups, it can cause a short-term drop. Overall, the trend is clear: larger boards lead to worse decisions and longer move times.

10 Convolutional Neural Networks

A tensor is a multi-dimensional array used to represent data in neural networks. In Tic-Tac-Toe, the board state is converted into a tensor — numbers that represent X, O, and empty positions — so the CNN can process it and detect patterns such as rows, columns, and diagonals. The CNN works step by step. The input is the game board, converted into a tensor. Then the CNN applies convolution layers. These layers use small filters that slide over the board, looking at small areas like 2×2 or 3×3 regions, to automatically detect local patterns. After that, pooling layers reduce the size of the data while keeping the most important information. The model repeats this process — convolution → pooling → convolution → pooling — so it can learn more and more complex patterns step by step. Finally, all the extracted features go into a fully connected layer, which combines everything together and produces a value score as the output. This value score replaces the traditional evaluation function in Minimax. Instead of using handcrafted rules, the CNN learns by itself how good a board position is. The CNN is trained using self-play data. First, training data is prepared through self-play. Then, the CNN takes a board state as input and produces a predicted value. Next, the loss is computed to measure how far the prediction is from the target. After that, backpropagation updates the model's weights. This process is repeated until the model improves.

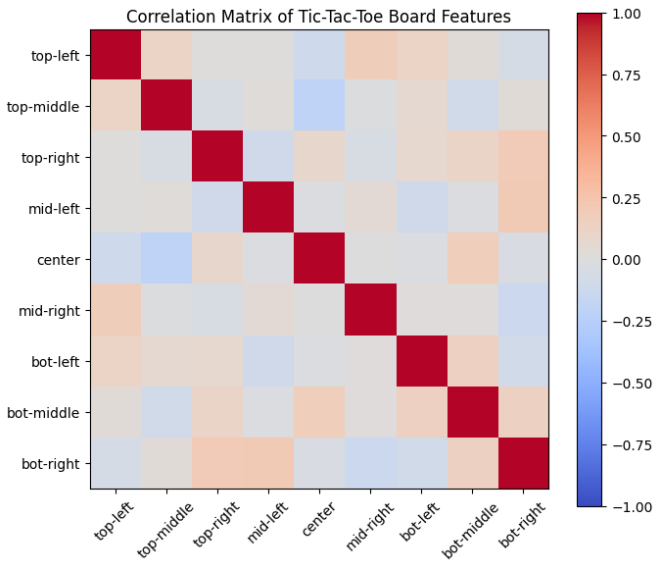

Fig. 12. Correlation matrix of Tic-Tac-Toe board features.

Some board positions are related to each other. Positions such as the center and corners are connected because of game rules and player strategies, meaning the board features are not fully independent.

```
board_sizes = [3, 4, 5, 6]

#tune these to keep runtime reasonable
shallow_depths = {
    3: 4,
    4: 4,
    5: 4,
    6: 4
}

reference_depths = {
    3: 5,
    4: 5,
    5: 5,
    6: 5
}

states_per_size = {
    3: 30,
    4: 30,
    5: 30,
    6: 30
}

avg_times = []
error_rates = []
```

Fig. 13. CNN-related network diagram.

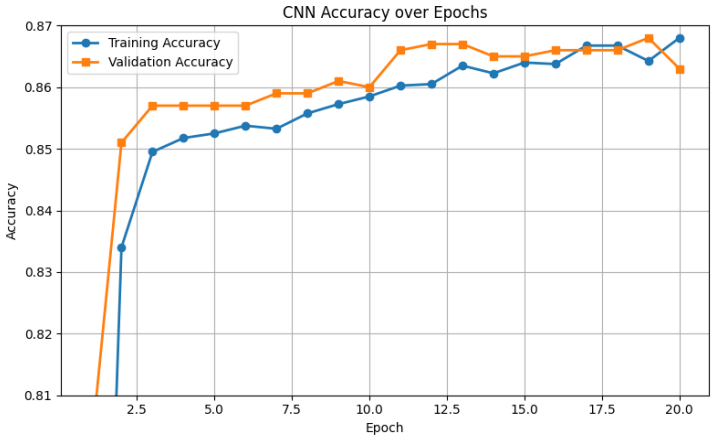

Fig. 14. CNN accuracy over epochs.

At the beginning, both training and validation accuracies increase, indicating that the model is learning. In later epochs, training accuracy becomes slightly higher than validation accuracy, indicating a slight overfitting. The model learns the training data well but does not generalize perfectly to unseen data.

11 CNN Does Not Improve Time Complexity

CNN evaluation does not directly reduce the main time complexity of Minimax. The search tree still exists, and the CNN is still called inside the tree search algorithm. Therefore, CNN evaluation gives better quality scores than traditional heuristics, it does not reduce the cost of searching the tree. It may reduce actual computation by evaluating positions better and helping prune more nodes, but the exponential growth of the search tree remains.

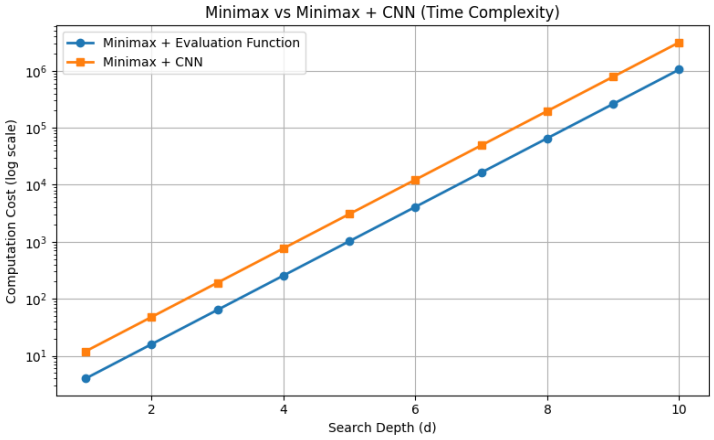

Fig. 15. Minimax vs Minimax + CNN (time complexity).

12 Deep Neural Network (DNN) Policy Network

The DNN policy network directly predicts the next move without searching the game tree. The input is the board state, represented as an 18-dimensional binary vector encoding the positions of X and O (9 for X, 9 for O). The output layer uses a softmax function (sum of possbility are 1) to produce a probability for each possible move, and the move with the highest probability is selected. This approach is called a policy network because it predicts the best next move directly from the board state.

The key difference between this approach and the Minimax approach is that Minimax is based on search and logic, while the policy network is based on learning patterns from data, in other woords, DNN drops the search tree. Both methods are useful in their own ways, but they reach their results differently.

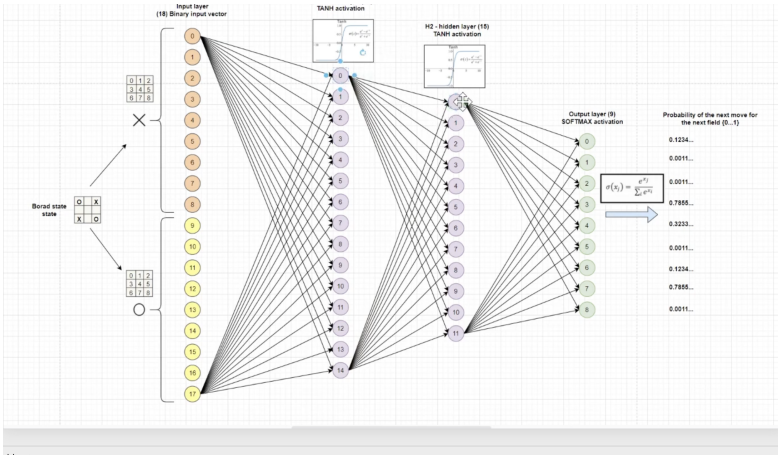

Fig. 16. DNN policy network architecture.

13 Weakness of DNN

The DNN policy network does not guarantee the best move. Because it makes decisions based on learned patterns instead of exploring all possible future moves, it can make mistakes. The model also depends on the quality of training data, so a model trained on small boards will not work well for large boards. This makes the cold start problem difficult to handle. The policy network can predict moves much faster than traditional methods, but the result is not always correct.

14 Absolute Strategy

For 3×3 Tic-Tac-Toe, there are known effective strategies: taking the center, blocking the opponent, preventing forks, and using corner strategy. However, as the board grows to 5×5, 6×6, or $n \times n$, the number of possible game states grows very fast, and there is no simple absolute strategy that works across all sizes.

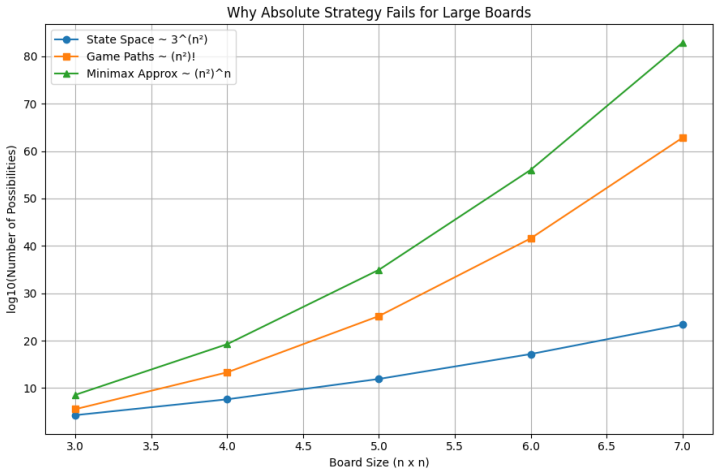

Fig. 17. Why absolute strategy fails for large boards.

The absolute strategy only works well for small boards like 3×3, and perhaps 4×4. When the board becomes bigger, computing or applying an absolute strategy becomes too expensive.

15 Failure

Handmade evaluation rules are simple, so they cannot understand complex patterns well, and they may not perform well in more complicated board situations. CNN-based evaluation can learn patterns from data, but it does not guarantee optimal decisions and depends heavily on the quality of the training data. Perfect Minimax can provide the best answers for small boards, but it does not scale well to larger boards due to high time complexity. The absolute strategy only works well for small board sizes such as 3×3, and possibly 4×4. However, when the size of the board increases (such as 5×5, 6×6, or n×n), the number of possible state of the game grows very quickly, making it too difficult to compute or apply an absolute strategy effectively.

16 Time Growth Across Board Sizes and Silly Decisions

As the board size increases from 3×3 to $n \times n$, two problems become clear: the computer takes much longer to decide, and it starts to make silly decisions.

On a 3×3 board, Minimax can check every possible future move. The average move time is only 0.002 seconds. Because the computer sees the full game tree, it always picks the best move and never makes a silly decision.

On a 4×4 board, the number of possible game states grows much larger. The average move time increases to 0.044 seconds. The computer can still search most of the tree, but it starts to take noticeably longer.

On a 5×5 board, the search tree becomes very large. The average move time jumps to 0.476 seconds. At this size, the computer can no longer check every possible move in a reasonable time. Because of this, it may miss important threats or winning moves, and silly decisions begin to appear. For example, the computer may fail to block the opponent from winning, or it may place a piece in a position that has no strategic value.

On a 6×6 board, the problem becomes even worse. The average move time reaches 2.214 seconds. The search tree is now so large that the computer spends most of its time exploring branches that do not lead to good moves. As a result, silly decisions happen more often. The computer may ignore an obvious winning move, or allow the opponent to build a long unblocked sequence without any response.

For an $n \times n$ board, the time complexity of Minimax is $O(b^m)$, where b is the number of possible moves at each step and m is the maximum depth of the game tree. As n grows, both b and m increase, so the total search time grows exponentially. This means the computer cannot finish its search before a time limit, and it is forced to make decisions based on an incomplete search. The result is longer move times and more silly decisions.

Table 1. Average move time vs board size (Minimax).

Board Size	Average Move Time (s)
3×3	0.002
4×4	0.044
5×5	0.476
6×6	2.214

16.1 Number of Nodes Expanded

As the board size increases, the number of nodes expanded by Minimax grows rapidly. Table 2 shows the number of nodes expanded for each board size.

Table 2. Number of nodes expanded vs board size (Minimax).

Board Size	Nodes Expanded
3×3	$\sim 10^3$
4×4	$\sim 10^5$
5×5	$\sim 10^7$
6×6	$\sim 10^9$

Each time the board size increases by one step, the number of nodes expanded grows by a large factor. This is because the branching factor b increases with board size, and Minimax must explore $O(b^m)$ nodes in total. For a 3×3 board, the full tree is small enough to search completely. For a 6×6 board, the number of nodes becomes so large that the search takes several seconds even for a single move.

16.2 Depth vs Accuracy

Another way to study this trade-off is to change the search depth and measure the decision error rate. Table 3 shows how increasing the depth affects accuracy.

Table 3. Search depth vs decision error rate.

Search Depth	Decision Error Rate (%)
2	65%
3	45%
4	30%
5	25%

As search depth increases, the computer can see further into the future, so it makes fewer silly decisions and the error rate goes down. However, increasing the depth also increases the number of nodes expanded, which means the move time becomes longer. This shows a clear trade-off: higher depth gives better decisions but costs more computation time. A depth of 2 or 3 is fast but makes more mistakes. A depth of 4 or 5 is more accurate but takes much longer, especially on larger boards.

In summary, as the board size grows, the computer takes exponentially longer to decide, and the quality of its decisions becomes worse. This shows why pure Minimax is not a practical solution for large boards.

17 Conclusion

This report discusses a few methods of building an intelligent Tic-Tac-Toe player: the Minimax algorithm, Alpha-Beta Pruning, Terminal Test and Utility, Depth-Limited Search, Heuristic Evaluation, CNN-based Evaluation, and DNN Policy Networks. All these approaches provide insight into how an intelligent agent can use different methods to make good decisions by doing a sequence of actions to achieve the goal state.

The most important conclusion is that checking every possible situation gives the best move but does not scale to larger boards. Minimax works best on small boards; Alpha-Beta Pruning improves time complexity but does not change exponential growth; heuristic evaluation reduces time but cannot handle complex large-board patterns; CNN-based evaluation learns patterns from data but is

still used inside the search tree; DNN policy networks make very fast decisions without searching the tree but do not guarantee the best move.

The progression of this project moves from exact search to learning-based decision making:

Minimax → Alpha-Beta Pruning → Terminal Test and Utility → Depth-Limited Search → Cutoff Test and Evaluation Function → CNN Evaluation → DNN Policy Network

For future work, I plan to prepare better training datasets and compare different deep learning models such as Recurrent Neural Networks and Reinforcement Learning models. Another direction would be to compare value networks and policy networks under the same conditions on the same board sizes.

References

1. S. Sridhar, "Classification of Tic-Tac-Toe Game Using Deep Neural Network," *International Journal of Scientific Research in Science, Engineering and Technology*, vol. 9, 2021, doi: 10.32628/IJSRSET219353.
2. S. Korbas, "AI playing Tic-Tac-Toe game – deep learning and neural network – part 1," YouTube, 2021. [Online]. Available: `https://www.youtube.com/watch?v=dFQB_9tWJmA`
3. S. H. Alkaraz, E. El-Seidy, and N. S. Morcos, "Tic-Tac-Toe: Understanding the Minimax Algorithm," *Journal of Game Theory*, vol. 9, no. 1, pp. 1–7, 2020, doi: 10.5923/j.jgt.20200901.01.
4. A. Marasco and S. Givigi, "Can AlphaZero Master Human Concepts in Tic-Tac-Toe?," in *Proc. IEEE Int. Conf. Systems, Man, and Cybernetics (SMC)*, Vienna, Austria, Oct. 2025, doi: 10.1109/SMC58881.2025.11343571.
5. N. Anand and A. Hussain, "Development of an AI-Based Agent for Tic-Tac-Toe Game Applications," *Journal of Science, Computing and Engineering Research*, vol. 8, no. 9, Sep. 2025, doi: 10.46379/jscer.2025.080606.
6. K. Sun, X. Wang, X. Miao, and Q. Zhao, "A review of AI edge devices and lightweight CNN and LLM deployment," *Neurocomputing*, vol. 614, Jan. 2025, Art. no. 128791, doi: 10.1016/j.neucom.2024.128791.
7. Y. Huang, S. Sun, X. Duan, and Z. Chen, "A Study on Deep Neural Networks Framework," in *Proc. IEEE Advanced Information Management, Communicates, Electronic and Automation Control Conf. (IMCEC)*, Xi'an, China, Oct. 2016, doi: 10.1109/IMCEC.2016.7867471.